

Guía Explicativa de Código C++

Sistema de Turnos Médicos | Librerías, Manejo de Archivos (GestorArchivo.h) y Validaciones (Validaciones.h)

1. Explicación de las Librerías Importadas en C++

En C++, una **librería** (o cabecera `#include`) contiene código preescrito por el lenguaje que nos brinda herramientas y funciones listas para usar. En este proyecto utilizamos las siguientes librerías estándar:

Librería	¿Qué significa el nombre?	¿Para qué sirve y qué hace en el programa?
<code><iostream></code>	Input / Output Stream	Permite la entrada y salida estándar por consola. Proporciona los objetos <code>std::cout</code> (para imprimir texto en pantalla) y <code>std::cin</code> (para leer lo que el usuario escribe en el teclado).
<code><fstream></code>	File Stream	Es la librería fundamental para trabajar con archivos en disco . Proporciona clases como <code>ifstream</code> (para leer archivos) y <code>ofstream</code> (para escribir en archivos).
<code><string></code>	String (Cadena de texto)	Proporciona el tipo de dato <code>std::string</code> , que permite manipular cadenas de texto dinámicas (nombres, apellidos, especializaciones) de forma muy sencilla.
<code><sstream></code>	String Stream	Permite tratar cadenas de texto como si fueran flujos de datos. Es clave para separar textos divididos por delimitadores (como el punto y coma <code>;</code>) usando <code>std::stringstream</code> y <code>std::getline</code> .
<code><iomanip></code>	Input / Output Manipulators	Nos brinda formateadores visuales para la consola, como <code>std::setw()</code> (define el ancho de las columnas de una tabla), <code>std::left</code> (alineación a la izquierda) y <code>std::fixed</code> / <code>std::setprecision()</code> (formato numérico).

Librería	¿Qué significa el nombre?	¿Para qué sirve y qué hace en el programa?
<code><limits></code>	Limits	Nos permite consultar los límites de capacidad de los tipos de datos del sistema. Lo utilizamos para vaciar por completo el buffer del teclado con <code>numeric_limits<streamsize>::max()</code> .
<code><cstdlib></code>	C Standard Library	Proporciona funciones para interactuar con el sistema operativo, tales como <code>system("cls")</code> que limpia la pantalla de la consola.

2. Conceptos Fundamentales: `ifstream` vs `ofstream`

Para entender el almacenamiento en disco, primero debemos comprender qué es un **Stream (Flujo)**: en programación, un flujo es un canal de transporte continuo de datos entre la memoria RAM del programa y un destino (la consola o un archivo en el disco duro).

OFSTREAM `ofstream` (Output File Stream)

¿Qué es? Es un flujo de salida dirigido hacia un archivo en el disco.

¿Para qué sirve? Para **GUARDAR o ESCRIBIR** datos en un archivo de texto.

Analogía: Funciona igual que `cout`. Mientras que `cout << "Hola";` envía el texto a la pantalla de la consola, `archivo << "Hola";` envía el texto al archivo `.txt` en el disco duro.

```
ofstream archivo("pacientes.txt"); // Abre o crea el archivo pacientes.txt
if (archivo.is_open()) {
    archivo << "1;Juan;Perez;12345678;OSDE\n"; // Escribe una línea
    archivo.close(); // Cierra el archivo liberando el disco
}
```

IFSTREAM `ifstream` (Input File Stream)

¿Qué es? Es un flujo de entrada proveniente de un archivo en el disco.

¿Para qué sirve? Para **CARGAR o LEER** datos desde un archivo de texto hacia la memoria RAM.

Analogía: Funciona equivalente a `cin`. Mientras que `cin >> variable;` lee desde el teclado, `getline(archivo, linea);` lee una línea completa proveniente del archivo `.txt`.

```
ifstream archivo("pacientes.txt"); // Abre el archivo en modo lectura
string linea;
if (archivo.is_open()) {
    while (getline(archivo, linea)) { // Lee línea por línea hasta el final (EOF)
        cout << "Línea leída: " << linea << endl;
    }
    archivo.close();
}
```

3. Funcionamiento de `GestorArchivo.h` Paso a Paso

El archivo `GestorArchivo.h` administra la persistencia de datos. Trabaja en conjunto con la ****Lista Doblemente Enlazada**** (memoria RAM) y los ****archivos de texto**** (disco rígido).

A. Serialización y Deserialización

- **Serialización (`toFileString()`):** Convierte un objeto en memoria (ej. Paciente) a una sola cadena delimitada por punto y coma (;).
Ejemplo: `id = 1, nombre = "Juan", apellido = "Perez"` → `"1;Juan;Perez;12345678;OSDE"`.
- **Deserialización (`fromFileString()`):** Proceso inverso. Toma una línea de texto del archivo y utiliza `std::stringstream` junto a `getline(ss, fragmento, ';')` para extraer valor por valor y reconstruir el objeto.

B. Proceso de Guardado (RAM → Disco)

1. Se crea una instancia de `ofstream archivo("pacientes.txt");`.
2. Se verifica que el archivo se haya abierto correctamente mediante `is_open()`.
3. Se inicia un puntero auxiliar en el primer nodo de la Lista Doblemente Enlazada:
`NodoDoble<Paciente>* aux = lista.getCabeza();`
4. Mientras el puntero `aux` no sea `NULL`:
 - Se llama a `aux->dato.toFileString()` para obtener el texto.
 - Se escribe en el archivo con el operador `<< "\n"`.
 - Se avanza el puntero al nodo posterior: `aux = aux->siguiente;`

- Se ejecuta `archivo.close();` para asegurar que todos los bytes se guarden físicamente en el disco.

C. Proceso de Carga (Disco → RAM)

- Se vacía la Lista Doble existente en la memoria RAM con `lista.vaciar()` para no duplicar datos.
- Se abre el archivo con `ifstream archivo("pacientes.txt");`.
- Se ejecuta un bucle `while (getline(archivo, linea))` que lee el archivo línea por línea hasta el final.
- Cada línea leída se transforma en un objeto mediante `Paciente p = Paciente::fromFileString(linea);`.
- El objeto reconstruido se inserta al final de la Lista Doble con `lista.agregarNodoAlFinal(p);`.
- Se cierra el archivo con `archivo.close();`.

4. Funcionamiento de `Validaciones.h` Paso a Paso

El módulo `Validaciones.h` protege la aplicación contra errores de entrada del usuario y mantiene limpia la consola visualmente.

El problema habitual en C++ con `std::cin`

Cuando la consola pide un entero (`int opcion; cin >> opcion;`) y el usuario por error escribe letras como `"hola"`, ocurre lo siguiente:

- El flujo `cin` entra en un estado de error grave llamado **failbit**.
- El texto no válido (`"hola"`) se queda atascado para siempre en la memoria interna del teclado (buffer de entrada).
- En un bucle `do-while`, el programa intentará leer de nuevo en la siguiente vuelta, volverá a encontrar `"hola"`, volverá a fallar y entrará en un **bucle infinito incontrolable**.

La Solución Implementada en `Validaciones.h`

```
static int ingresarEntero(const string& mensaje) {
    int valor;
    while (true) {
        cout << mensaje;
        if (cin >> valor) { // INTENTA LEER EL ENTERO
```

```

        // ¡ÉXITO! Se leyó el número correctamente
        cin.ignore(numeric_limits::max(), '\n'); // Limpia el sobrante
        return valor;
    } else {
        // ¡ERROR! El usuario ingresó letras o caracteres inválidos
        cout << "[!] Entrada inválida. Por favor ingrese un número entero.\n";
        cin.clear(); // 1. REPARA Y RESTABLECE LAS BANDERAS DE ERROR DE CIN
        cin.ignore(numeric_limits::max(), '\n'); // 2. BORRA EL TEXTO CORRUPTO DEL BUFFER
    }
}
}
}

```

Explicación de las dos líneas clave:

- `cin.clear();` : Resetea el indicador de error de C++ para que el flujo `cin` vuelva a estar activo y funcional.
- `cin.ignore(numeric_limits<streamsize>::max(), '\n');` : Elimina del buffer del teclado absolutamente todos los caracteres hasta encontrar el salto de línea (Enter). Con esto el buffer queda completamente limpio para la siguiente oportunidad.

Formato de Pantalla: `limpiarPantalla()`

```

static void limpiarPantalla() {
#ifdef _WIN32
    system("cls"); // Comando propio de la consola de Windows
#else
    system("clear"); // Comando propio de Linux / macOS
#endif
}

```

Utiliza directivas de compilación condicional (`#ifdef _WIN32`) para garantizar que el programa funcione y limpie la pantalla limpiamente tanto en computadoras con Windows como con Linux o Mac.